

🔗 Generando una Frase Semilla BIP39 con un Dado de 20 Caras — Edición Extendida: 12 y 24 Palabras

Una frase semilla mnemónica BIP39 proporciona la entropía maestra de tu billetera de Bitcoin. Aunque el software puede generar esa aleatoriedad, el uso de dados físicos ofrece una fuente de entropía transparente y verificable, que no depende de ningún generador electrónico de números aleatorios. Esta edición extendida cubre el procedimiento completo de 12 palabras, el procedimiento de 24 palabras y el atajo de autocompletado de la última palabra que ofrecen varias herramientas de semilla. Usa la misma tabla de consulta (al dorso).

📖 ¿Por qué entropía verificable? — el incidente de julio de 2026

En julio de 2026, más de mil BTC fueron vaciados en minutos de billeteras cuyas semillas habían sido creadas por dispositivos Coldcard con un defecto de firmware existente desde 2021: la generación de la semilla usaba silenciosamente un PRNG predecible de software en lugar del RNG de hardware, entregando mucha menos entropía de la prometida. El fabricante corrigió rápidamente — y su propio aviso oficial señala que las semillas generadas con 50 o más tiradas honestas de dados nunca estuvieron en riesgo. La lección no trata de una marca: **toda entropía electrónica es invisible para el usuario**. No puedes mirar un chip y ver si su aleatoriedad está sana — pero sí puedes mirar un dado. Con este tutorial, cada bit de tu semilla proviene de tiradas que verificas con tus propios ojos.

📊 Entendiendo la Estructura de la Tabla

La tabla de referencia está organizada como un sistema de consulta tridimensional. Cada una de las 2048 palabras del estándar BIP39 se identifica de forma única mediante tres coordenadas, que llamaremos D1, D2 y D3. La primera coordenada (**D1**) selecciona una de las **8 secciones** de la página, la segunda (**D2**) selecciona una de las **16 filas** dentro de esa sección, y la tercera (**D3**) selecciona una de las **16 columnas**. Juntos, estos tres valores identifican exactamente una palabra de la tabla. Las palabras de la semilla son siempre las del estándar BIP39 en inglés, aceptadas por prácticamente todas las billeteras.

🎲 El Procedimiento de Tiradas (12 Palabras)

Para cada una de las primeras 11 palabras de tu frase semilla, debes obtener tres resultados válidos del dado. Tira el dado y registra el resultado solo si muestra un valor entre 1 y 16 (inclusive). Si el dado muestra 17, 18, 19 o 20, **descarta esa tirada** y vuelve a tirar hasta obtener un resultado válido. Repite este proceso hasta tener tres números válidos (D1, D2, D3) para la palabra actual.

💡 Ejemplo de Generación de una Palabra

Supón que estás generando la quinta palabra de tu semilla. Tiras el dado y obtienes: 19 (inválido, tira de nuevo), 3 (válido, D1 = 3,4), 1 (válido, D2 = 1), 20 (inválido, tira de nuevo), 11 (válido, D3 = 11). Tus coordenadas son ((3,4), 1, 11). Navega hasta la sección 3,4 de la tabla, busca la fila 1 y luego la columna 11 para obtener tu palabra: **candy**.

✍️ La 12ª Palabra

Para generar la última palabra de tu semilla BIP39, repite el procedimiento para las 2 primeras coordenadas igual que hiciste con las primeras 11 palabras, seleccionando así la sección y la fila de la tabla. En este punto, existe exactamente una única palabra en la fila seleccionada (entre 16 alternativas) que formará una semilla BIP39 válida. La columna de la última palabra **no** se determina tirando el dado, sino por ensayo y error en el asistente de entrada de semilla de tu billetera — preferiblemente en un dispositivo *air gapped*: rechazará las 15 candidatas inválidas y aceptará la única válida.

💡 Ejemplo de Generación de la Última Palabra

Supón que tus primeras 11 palabras son, en orden: *«never, use, this, example, because, private, key, secret, need, phrase, y true»*. Ahora generarás tu 12ª palabra. Tiras el dado y obtienes: 18 (inválido, tira de nuevo), 12 (válido, D1 = 11,12), 9 (válido, D2 = 9). Con eso, tu 12ª palabra ya está determinada: prueba una por una las palabras de la fila 9 de la sección 11,12 y verás que la palabra de la columna 14, **random**, (y solo ella), junto con las 11 primeras, en este orden, forma una semilla BIP39 válida:

1.never 2.use 3.this 4.example 5.because 6.private 7.key 8.secret 9.need 10.phrase 11.true 12.random. 🟢

Comprobación (p.ej. en herramienta de autocompletado): palabras válidas de la sección 11,12, una por fila (1–16, en orden): payment, people, planet, pole, possible, prize, pulp, quality, random, raw, relief, result, return, room, royal, say.

🔗 Atajo: Asistentes con Autocompletado de la Última Palabra

Varias herramientas de semilla completan la última palabra por ti: dadas todas las palabras anteriores, listan exactamente las candidatas que hacen válido el *checksum* — por ejemplo el flujo *final word* de SeedSigner, la calculadora de última palabra de SeedPicker, el BIP39 Recoverer offline de Coinplate o la página BIP39 de Ian Coleman (usada offline). Para una semilla de 12 palabras existen siempre exactamente **128 últimas palabras válidas: precisamente una en cada una de las $8 \times 16 = 128$ filas** de la tabla. Así que, en vez del ensayo y error columna por columna: tira D1 y D2 como de costumbre y toma, entre las candidatas del asistente, **la que esté en tu sección y fila tiradas**. La misma palabra, la misma entropía, menos tecleo. Y no es casualidad de nuestro ejemplo: *cualquiera* que sean las primeras 11 palabras, fijados D1 y D2, una y solo una palabra de esa fila satisface el checksum — las pruebas de sanidad del repositorio lo verifican mecánicamente sobre prefijos aleatorios.

24 Palabras

Para una semilla de 24 palabras (256 bits de entropía), genera las palabras 1–23 exactamente como las 11 primeras de arriba: tres tiradas válidas cada una ($23 \times 11 = 253$ bits). Para la **24ª palabra, tira solo D1**, seleccionando la sección (3 bits más — 256 en total). La fila y la columna quedan entonces fijadas por el *checksum* de 8 bits: **exactamente una palabra en toda tu sección de 256 palabras** completa una semilla válida. Encuéntrala de una de estas dos formas:

- **Ensayo y error** en el asistente de la billetera, a lo largo de la sección tirada (hasta 256 intentos — tedioso, pero totalmente offline); o
- **Autocompletado de la última palabra:** un asistente como los de arriba ofrecerá exactamente **8 candidatas válidas — una por sección**. Tu tirada de D1 elige la sección; toma la candidata que esté en ella.

Igualmente garantizado, no coincidencia: *cualquiera* que sean las primeras 23 palabras, fijado D1, una y solo una palabra de esa sección satisface el checksum — también verificado por máquina por las pruebas de sanidad del repositorio.

🔦 Ejemplo de 24 Palabras: la Frase del Ejemplo, Duplicada

Toma el ejemplo de 12 palabras de arriba y duplícalo para obtener las palabras 1–23; para la 24ª, supón que tiras: 17 (inválido, tira de nuevo), 11 (válido, D1 = 11,12). Entre las 8 candidatas del asistente (o por ensayo y error a lo largo de la sección 11,12), exactamente una palabra de esa sección — **polar**, fila 4, columna 12 — completa una semilla válida de 24 palabras:

```
never use this example because private key secret need phrase true random
never use this example because private key secret need phrase true polar ✓
```

Nota el sacrificio: la duplicación no puede conservar **random** como 24ª palabra, porque el checksum de 8 bits de los 256 bits completos difiere del checksum de 4 bits que hacía válida a **random** con 12 palabras. El checksum reclama la última palabra — y cae en **polar**, la única palabra de la *misma sección* (D1 = 11,12) que **random** cuya combinación con las 23 anteriores lo satisface. La lista completa de candidatas, una por sección: **because** (1,2), **critic** (3,4), **fresh** (5,6), **ice** (7,8), **limb** (9,10), **polar** (11,12), **spare** (13,14), **wolf** (15,16).

☰ Resumen

- Tira un dado de 20 caras. **Descarta** los resultados **17, 18, 19 o 20**
- Cada tirada válida (de 1 a 16 inclusive) especifica una coordenada de una palabra
- Cada una de las **8 secciones (D1)** corresponde a **2** resultados válidos posibles; cada una de las **16 filas (D2)** y **16 columnas (D3)**, a **1**
- **12 palabras:** 11 palabras tiradas por completo y luego solo D1 + D2; **35 tiradas válidas** producen los **128 bits** completos. La última palabra: ensayo y error en la fila, o la candidata del asistente en tu fila tirada (existen **128** candidatas válidas, una por fila)
- **24 palabras:** 23 palabras tiradas por completo y luego solo D1; **70 tiradas válidas** producen los **256 bits** completos. La última palabra: ensayo y error en la sección, o la candidata del asistente en tu sección tirada (existen **8** candidatas válidas, una por sección)
- Los bits de *checksum* (4 u 8) corren siempre por cuenta de la billetera — nunca se tiran

⚠️ Advertencias de Seguridad

- Nunca uses este ejemplo de arriba porque las claves privadas, para ser secretas, necesitan que su frase sea verdaderamente aleatoria
- Realiza este procedimiento en un lugar privado. No fotografíes ni registres digitalmente tus tiradas ni resultados intermedios. Escribe tu frase semilla final en papel y guárdala en un lugar seguro
- Usa las herramientas de autocompletado únicamente en dispositivos *air gapped* de confianza — ven tu semilla entera. Para máxima seguridad, escribe tu frase semilla únicamente en dispositivos *air gapped* de confianza y de tu propiedad
- Si sospechas que una de tus semillas fue generada por software o firmware defectuoso (como en el incidente de julio de 2026), genera una semilla nueva con este método y transfiere tus fondos a ella: actualizar el firmware **no** repara una semilla débil ya existente

📄 Para Saber Más

Este tutorial es obra de **Yuri da Silva Villas Boas** — autor de la BIP-450 (Formosa) y creador del protocolo Great Wall. Repositorio oficial (fuentes, PDF en 4 idiomas, verificación automática): github.com/Yuri-SVB/BTC-D20 — o escanea el código QR al lado.

Artículo corto: por qué un aviso discreto era imposible — Kerckhoffs y el incidente de julio de 2026: [posts/kerckhoffs-lemma-coldcard.mdn](https://posts.kerckhoffs-lemma-coldcard.mdn) el repositorio

Más tutoriales, cursos y comunidad: yuri-svb.github.io — y síguenos en Twitter/X, Nostr y Telegram

Great Wall: Generar una semilla fuerte es solo el primer paso; protegerla contra robo y coerción es el resto. Great Wall es nuestro protocolo de software libre para autocustodia resistente a la coerción: github.com/Yuri-SVB

